ZAVIONIX / ETHOS

Heli
User Manual

An electric-helicopter dashboard combining battery information with RPM, temperature and a flight-mode source.

Widget version 3.1.12 | English | 15 October 2026

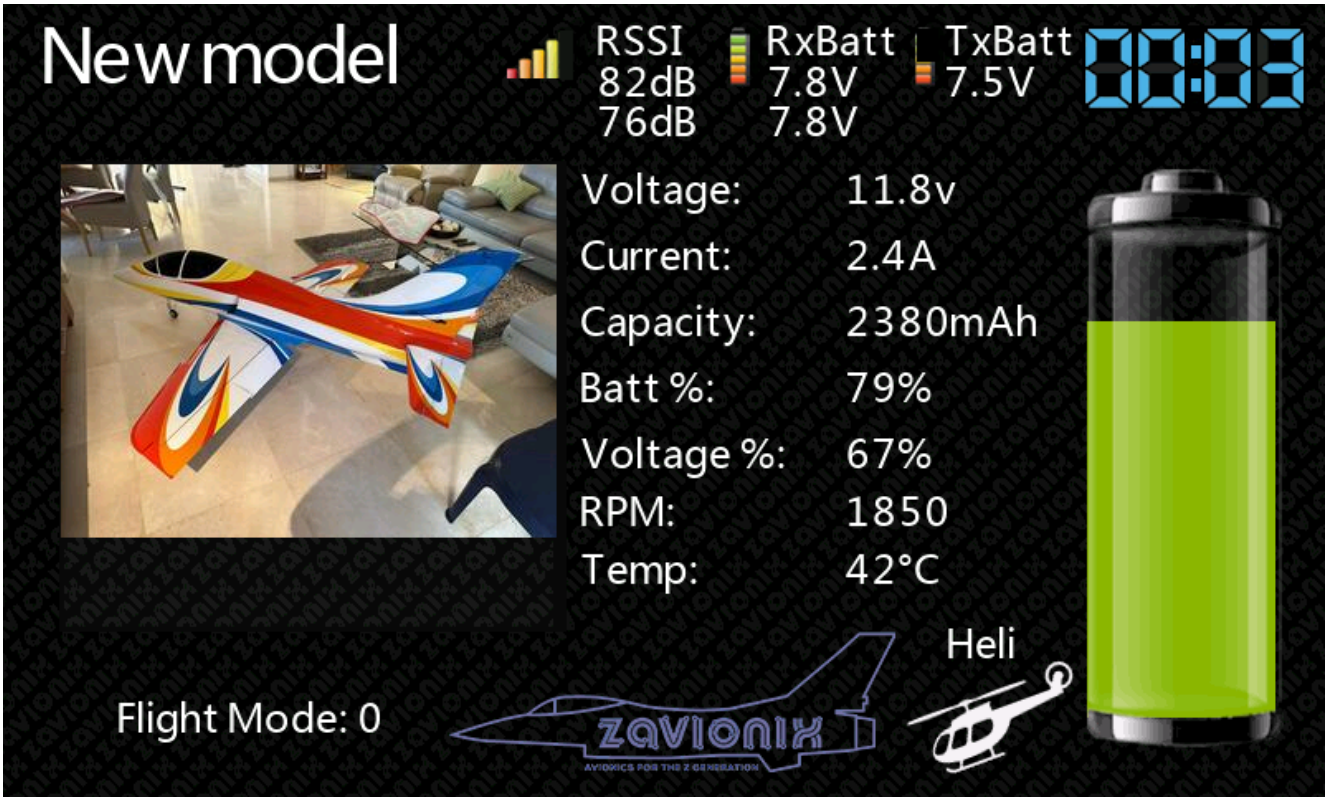


Figure 1. Actual ETHOS simulator rendering with simulated sensor data. The example is not a flight record or a recommended operating setup.

Read this guide in order for first installation. Use the settings and sensor reference when adapting an existing model.

1. Before you begin

An electric-helicopter dashboard combining battery information with RPM, temperature and a flight-mode source.

You need a compatible color-screen ETHOS transmitter or simulator, the complete widget folder, and the relevant telemetry sources already available in the model. These captures use the VS Code-cached X20S ETHOS 26.1.3 simulator engine at 800 x 480. Other screens may arrange or scale the content differently.

This manual covers installation, configuration, every captured widget-specific form field, normal use and troubleshooting. Menu labels are written as they appear in this version. Numeric examples are demonstration settings, not model-specific recommendations.

Guide contents

2. Install and add the widget

3. Configure telemetry and read the display

4. Operating guide

5. Settings and sensor reference

6. Troubleshooting and verification

7. Screenshot provenance and version record

2. Install and add the widget

1. Back up the model and existing widget folder. Download the Heli repository ZIP from https://github.com/i3dm/Heli (Code / Download ZIP) or use the appropriate version of the repository.

2. Extract the archive and copy the whole src/Heli folder into the radio storage /scripts directory, producing /scripts/Heli/main.lua. Keep the companion Lua files, images and audio in their supplied relative locations. Do not copy only main.lua.

3. Restart ETHOS or reload Lua through the facilities supported by your firmware. If upgrading, use one complete version of the package rather than mixing old and new module files.

4. Open Configure screens (DISP in the simulator). Add or select a screen and choose a suitable layout; for a full-screen view, long-press its tab and use Change layout. Tap the widget area and select Zavionix Heli from Widget.

5. Open the widget configuration, set the model-specific parameters, and assign telemetry sources. Use the back arrow to return to the dashboard. Reopen the settings and confirm the assignments persist.

Use only the production widget folder on a real transmitter. Documentation sensor fixtures and simulator radio copies are not part of the installation.

3. Configure telemetry and read the display



Figure 2. Widget configuration in ETHOS. Scroll down to reach the remaining settings and source selectors; the reference lists fields below the visible area as well.

Power the intended telemetry hardware in a controlled setup and discover its sensors in the ETHOS model telemetry page. Confirm each value and unit there before assigning it in the widget. Receiver 1/2 and engine left/right inputs must refer to the intended physical source.

A missing value or a No ... label normally means an unassigned or unavailable source. A plausible but incorrect value often means the wrong source, unit or scale. Inspect the source in ETHOS telemetry first; do not compensate for a bad assignment by changing an unrelated capacity or alert setting.

Where displayed, RSSI and RxBatt come from your selected receiver sources; TxBatt comes from the transmitter. The model name and picture come from ETHOS model settings. A missing model image does not imply a telemetry failure. The example photograph is specific to the test model.

4. Operating guide

Battery setup and interpretation

Set each full battery capacity before interpreting a percentage. The capacity-based estimate is (configured capacity minus consumed mAh) / configured capacity x 100. For example, a 3000 mAh battery with 620 mAh consumed has 2380 mAh remaining, or about 79%. A consumption sensor reporting remaining mAh will invert the meaning and produce wrong results.

After charging or replacing a battery, reset the upstream consumption counter using the sensor/ETHOS reset arrangement for that model. A widget restart is not evidence that the physical consumption counter was reset. Check that the new-flight consumed reading is correct before use.

Assign matching voltage, current and consumption sources to each battery. Never infer that battery 2 is healthy from a duplicated battery 1 source. If the hardware supplies only one current measurement, use a widget designed for that arrangement or interpret the missing channel accordingly.

Set the remaining-mAh alert above the reserve you intend to retain, enable the alert, and choose its repeat interval. Compare calculated remaining capacity with charge returned to the pack over several representative uses. A stale or disconnected sensor can make a plausible percentage unreliable.

Capacity percentage versus voltage percentage

Batt % follows consumed mAh. Voltage % uses measured pack voltage divided by Number Of Cells, then a chemistry-specific voltage table. These estimates can disagree because load, temperature, cell condition and voltage recovery affect the voltage estimate.

The illustrated 3-cell LiPo example is 11.8 V, 2.4 A and 620 mAh consumed from 3000 mAh. It displays about 79% by capacity and 67% by voltage. Choose the actual chemistry and series cell count; changing them only to make the percentages match hides a configuration error.

Helicopter telemetry

Assign RPM to the rotor or shaft speed you intend to monitor and configure the telemetry gearing accordingly. The example shows 1850 RPM, 42 C and flight-mode value 0. Assign the temperature probe to the component you wish to monitor and use the Flight Mode Sensor selector for the desired mode source.

The widget displays flight-mode telemetry; it does not select a flight mode or change a governor. Verify the displayed value in every flight mode while the model is safely inactive.

5. Settings and sensor reference

The following labels were read from the running widget configuration. Example numbers describe the documentation setup or initialized fields; they are not guaranteed factory defaults for a previously saved model. Source assignments and switches must be chosen for your model. Decimal values below use the units displayed in the form.

Parameters

Menu field	Choices / example / range	Purpose
Battery Type	Lipo; HV Lipo; Lion; LiFe	Select the actual cell chemistry. It selects the voltage-to-percentage lookup, not the charger settings.
Battery Capacity	Example 3000; range 1 to 50000	Full usable battery capacity in mAh. Enter the appropriate value for this battery; remaining capacity is this setting minus telemetry consumption.
Low Battery Alert	On / Off; example On	Enable or disable the named audible alert. Confirm the associated threshold and repeat interval; disabling an alert does not correct bad telemetry.
Batt Alert (mAh)	Example 1000; range 1 to 50000	Remaining-capacity alert threshold in mAh. It is not the consumed-capacity value.
Low Battery Alert Rate (s)	Example 5; range 1 to 10	Minimum repeat interval in seconds for the associated alert while its condition persists. The sound files must remain in the widget folder.
Number Of Cells	Example 3; range 1 to 14	Series cell count used to convert pack voltage to average cell voltage for the voltage-percentage estimate.

Source assignments

Source selector	Required value and interpretation
Voltage Sensor	Measured battery voltage in volts. Match battery 1/2 to its own current and consumed-capacity sources.
Current Sensor	Measured current in amperes. For a turbine, assign the protocol current channel only when supported by the selected ECU.
Consumption Sensor	Cumulative consumed capacity in mAh, not remaining capacity. The widget subtracts this from the configured full battery capacity.
RPM Sensor	Engine/rotor speed in RPM. Configure sensor gearing/pole count upstream so the source reports the intended shaft speed.
Temp Sensor	Temperature in degrees Celsius. Assign each numbered input to the intended probe and document its location.
Flight Mode Sensor	Source carrying the desired flight-mode value. The widget displays that source value; it does not change the active flight mode.
RSSI 1 Sensor	Receiver signal-strength source in dB. Keep receiver 1 and receiver 2 distinct; leave an unavailable second link unassigned.

Source selector	Required value and interpretation
RSSI 2 Sensor	Receiver signal-strength source in dB. Keep receiver 1 and receiver 2 distinct; leave an unavailable second link unassigned.
Rx Batt 1 Sensor	Receiver supply voltage in volts. This is separate from transmitter voltage and, where applicable, the main propulsion battery.
Rx Batt 2 Sensor	Receiver supply voltage in volts. This is separate from transmitter voltage and, where applicable, the main propulsion battery.

6. Troubleshooting and verification

Symptom	Checks
Widget absent from selector	Check /scripts/Heli/main.lua and all companion files. Restart ETHOS and inspect Lua messages. Make sure the archive did not create an extra nested folder.
Error after update	Restore a complete matching package version. Check available Lua memory and the exact error in the simulator/radio log; retain the model backup.
Missing / No sensor value	Inspect Model / Telemetry, power the sensor, rediscover if necessary, then reselect the correct source in the widget.
Value has wrong magnitude	Check units, decimals, ECU selection and upstream sensor scaling against a known reference. Do not change warning thresholds to conceal a scaling error.
Values do not change	Compare the original telemetry page. A retained or stale last value is not proof of an active link. Confirm fresh telemetry before use.
Text is clipped	Use a larger widget area/full-screen layout. Cells also has Font Size. Check the intended screen size and version.
No audible alert	Verify enable switch, threshold, repeat interval, telemetry validity, audio files and transmitter volume. Headless screenshots cannot verify the speaker.
Settings appear different	Confirm the selected model, widget instance and, for combined widgets, Widget Display. Each can have its own saved settings.

Model acceptance check

1. Reopen the saved model and verify every configured source and unit.

2. Compare the widget with the underlying telemetry at two or more known values. For twin/battery pairs, vary one side at a time.

3. Check resets, timer controls and any alert conditions relevant to your model in a controlled test. Observe the physical transmitter audio where alerts matter.

4. Verify behavior when a source becomes unavailable and after restoring it. Do not infer source-loss behavior from a static screenshot.

5. Record the final capacities, thresholds, scaling and calibration values with the model backup.

7. Screenshot provenance and version record

The screenshots were produced by the genuine ETHOS X20S 26.1.3 engine (a074fdfd) cached by the VS Code ETHOS extension, driven headlessly through the ethos-tools PR #2 runner. This validates Lua execution and rendering in that engine; it is not a claim that the VS Code webview or every supported transmitter was tested.

Documentation-only copies supplied genuine ETHOS sensor objects and assigned them to the widget. The rendering implementation was retained. Most inputs used DIY sensors refreshed with rawValue; Cells used injected S.Port cell frames to provide an actual multi-cell source. Demonstration settings were initialized independently of the copied model. These fixtures were not installed in the production widget repository.

The simulation method was informed by WingFlight's simulator sensor implementation. The example values are synthetic, not flight measurements. Static captures and configuration inspection do not establish calibrated measurement accuracy, full alert-state coverage, audible playback or physical control performance.

Widget: Heli 3.1.12. Source commit inspected: 4bc59592f7ecaf4f65d8f79eadeff0c42b789fea. Documentation edition: 15 October 2026. Repository: https://github.com/i3dm/Heli.

Source links

Widget source and updates

ETHOS simulator automation PR

WingFlight simulator sensor approach